



UNIWERSYTET VIZJA

School of Computer Science & Technologies

Field of study: Computer Science

**Uladzimir Kireyeu
45138**

Engineering Project “PartTrack”

Bachelor thesis written under supervision of
Krzysztof Rychlicki-Kicior

Warsaw, 2026

Basic Information

PartTrack	PartTrack — Web-based inventory management system for manufacturing stations
Project Goal	Manufacturing stations rely on a steady supply of spare parts and components to keep production running. When stock runs low and nobody notices in time, production can stall while waiting for a reorder to arrive. This problem is especially common in smaller facilities where inventory is still tracked in spreadsheets or by eye. PartTrack was built to solve this for small and medium-sized manufacturing teams. Each station has a dedicated delegate who logs stock levels after each shift. A background process watches all levels continuously and automatically emails the responsible people when a part is about to run out, factoring in whether a replacement is already on the way. This removes the dependency on manual checking and ensures the right people are notified before a shortage causes a problem. The system targets production managers and warehouse staff who need something more structured than a spreadsheet but do not want to pay for or maintain a large enterprise ERP system.
Short Project Description	PartTrack is a full-stack web application that allows manufacturing teams to track spare parts inventory across multiple stations in real time. Administrators manage stations, parts, users, and alert recipients through a browser interface. Station delegates log stock adjustments and record incoming shipments with tracking links. Each shipment is tracked

	from the moment it is ordered through to physical delivery, keeping the in-transit quantity visible at all times. An hourly background job checks every part against its minimum threshold. If combined stock and in-transit quantity falls below that threshold, an HTML email is sent to all configured recipients. To avoid alert fatigue, a progressive cooldown applies: immediate on first detection, then every 24 hours for up to three alerts, then weekly on Monday mornings. Alerts are suppressed automatically once stock recovers. The application runs entirely inside Docker containers and can be started with a single command.
Competition Analysis	Several commercial inventory management products exist in this space. The three most relevant are Sortly, inFlow, and Fishbowl. Sortly is a simple visual inventory tool for small teams with good mobile support and QR code scanning. However, it is a general-purpose asset tracker with no concept of production stations, shipment tracking, or threshold-based alerts. Pricing starts at \$49/month. inFlow is a cloud-based platform handling purchase orders, sales orders, and basic warehouse operations. Pricing ranges from \$89 to \$439/month. Its main limitation is that it is designed around finished goods and customer orders rather than maintaining minimum stock levels at production stations. Alert capabilities are limited and it lacks per-station access control. Fishbowl is the most feature-complete option, offering work order management,

	bill of materials, and QuickBooks integration. It is suited for full manufacturing resource planning but costs around \$329/month and comes with considerable setup complexity. Users consistently report a steep learning curve and an outdated interface. For a team that only needs stock level monitoring and alerts, it is over-engineered. Where PartTrack fits: None of the above combine station-scoped access control, in-transit alert suppression, and a shipment lifecycle in a self-hosted, zero-subscription package. PartTrack targets the gap between a spreadsheet and a commercial product — teams that need automation and accountability without an ongoing SaaS subscription.
List of Technologies	Backend: Node.js 20, Express 4, PostgreSQL 16, node-pg-migrate 8, jsonwebtoken 9, bcryptjs 3, express-validator 7, node-cron 4, nodemailer 8, Jest 29, supertest 7 Frontend: React 18, Vite 5, React Router 6 Infrastructure: Docker, Docker Compose, nginx (alpine)
Description of Technologies and Justification	<i>How the technologies fit together:</i> The system runs as three Docker containers. The frontend container serves the compiled React application via nginx and proxies /api/* requests to the backend, so only port 80 is exposed and no CORS configuration is needed. The backend runs Node.js, handling API requests, the hourly cron job, and email delivery. PostgreSQL runs in the third container with data persisted in a named volume. Migrations apply automatically on startup, keeping the schema in sync with

	the code. <i>Justification:</i> Node.js and Express — JavaScript across the full stack reduces context-switching. Express is minimal and unopinionated, which kept the project lean. PostgreSQL — the data model has genuine relational complexity: parts belong to stations, shipments and audit entries reference both parts and users, and cooldowns reset conditionally. PostgreSQL handles transactions, row-level locking, and complex queries reliably. SQLite would not have been appropriate here. node-pg-migrate — every schema change is a numbered migration file. The database can be rebuilt from scratch at any time and the change history is fully traceable in version control. JWT authentication — stateless tokens mean no server-side session storage. An 8-hour expiry matches a typical working shift. React with Vite — React is the most widely used UI library with a large ecosystem. Vite produces fast builds. No component library was used — plain CSS with custom properties keeps the bundle small and the design fully under control. Docker and Docker Compose — the entire stack starts with one command and behaves identically in development and production. No local Node.js or PostgreSQL installation is required on the host machine.
--	--

Key issues

Database design

The most deliberate schema decision was keeping stock quantity in a separate `stock_levels` table rather than as a column on `parts`. When two requests update the same part simultaneously, PostgreSQL can lock just the single stock row with `SELECT ... FOR UPDATE` without blocking reads of the part's name or threshold. Every write touching more than one table is wrapped in a transaction.

The `email_cooldowns` table stores one row per part with a `last_sent_at` timestamp and an `alert_count` integer. This is enough to implement the full progressive alert logic without any additional tables. Cooldown rows are deleted automatically when stock recovers, so the counter resets cleanly:

```
DELETE FROM email_cooldowns ec
USING parts p, stock_levels sl
WHERE ec.part_id = p.id AND sl.part_id = p.id
      AND sl.quantity + COALESCE((
        SELECT SUM(sh.quantity) FROM shipments sh
        WHERE sh.part_id = p.id AND sh.status = 'pending'
      ), 0) >= p.min_threshold
```

Atomic shipment delivery

Marking a shipment as delivered involves three writes: updating the shipment status, updating the stock quantity, and inserting an audit entry. If any one fails, the others must roll back:

```
await client.query('BEGIN');

await client.query(
  "UPDATE shipments SET status = 'delivered', delivered_at = now() WHERE id = $1",
  [shipmentId]
);
const { rows: [sl] } = await client.query(
  'SELECT quantity FROM stock_levels WHERE part_id = $1 FOR UPDATE',
  [partId]
);
const newQty = sl.quantity + shipment.quantity;
await client.query(
  'UPDATE stock_levels SET quantity = $1, updated_at = now() WHERE part_id
```

```

= $2',
  [newQty, partId]
);
await client.query(
  `INSERT INTO audit_log(user_id, part_id, old_quantity, new_quantity,
delta, note)
  VALUES($1,$2,$3,$4,$5,$6)`,
  [req.user.id, partId, sl.quantity, newQty,
  shipment.quantity, `Shipment #${shipmentId} delivered`]
);

await client.query('COMMIT');

```

The **FOR UPDATE** lock prevents a race condition where two deliveries arrive simultaneously and both read the same stale quantity before either has written the update.

Progressive alert cooldown

The decision of whether to alert for a given part is extracted into a single pure function:

```

function shouldAlert(r, now) {
  const inTransit = parseInt(r.in_transit);
  if (r.quantity + inTransit >= r.min_threshold) return false;
  if (!r.last_sent_at) return true;

  const hoursSinceLast = (now - new Date(r.last_sent_at)) / (1000 * 60 *
60);
  const count = parseInt(r.alert_count);

  if (count < 3) return hoursSinceLast >= 24;

  const isMonday8am = now.getDay() === 1 && now.getHours() === 8;
  return isMonday8am && hoursSinceLast >= 144;
}

```

The first check is in-transit suppression: if combined physical stock and pending shipments already cover the threshold, no alert is sent even if physical stock alone is below it. This prevents noise when a reorder is already in flight. The cooldown then

escalates: no delay on the first alert, 24-hour gaps for the next two, then weekly on Monday at 08:00.

Extracting this as a standalone function made all seven branches testable without a database, for example:

```
it('suppresses when qty + in_transit >= threshold', () => {
  expect(shouldAlert({ ...base, quantity: 2, in_transit: 3 },
    now)).toBe(false);
});
```

Authentication and input validation

Every protected route goes through a middleware that verifies the JWT and attaches the decoded payload to `req.user`. A second middleware layer checks the role for admin-only routes:

```
function requireAuth(req, res, next) {
  const header = req.headers.authorization;
  if (!header?.startsWith('Bearer '))
    return res.status(401).json({ error: 'Missing token' });
  try {
    req.user = jwt.verify(header.slice(7), process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ error: 'Invalid token' });
  }
}
```

`JWT_SECRET` is read from the environment only. The application calls `process.exit(1)` on startup if it is not set, so a misconfigured deployment fails immediately rather than silently accepting tokens at runtime.

All incoming request data is validated with `express-validator` before reaching any database query. Validators are declared inline with the route and sanitisers like `.toInt()` and `.trim()` run as part of validation, so by the time the handler body executes the values are already the correct types:

```
router.post(
 ('/:id/shipments',
```

```

requireAuth,
body('quantity').isInt({ min: 1 }).toInt(),
body('tracking_link').optional({ nullable: true }).trim().isURL(),
async (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty())
    return res.status(400).json({ error: errors.array()[0].msg });
  // req.body.quantity is already an integer here
}
);

```

Global toast notifications in React

User feedback is managed through a React context provider so any component can show a notification without prop drilling. The provider holds the toast state and exposes a `showToast` function via context:

```

const showToast = useCallback((msg, type = 'success') => {
  clearTimeout(leaveTimer.current);
  clearTimeout(removeTimer.current);
  setToast({ msg, type, leaving: false });
  leaveTimer.current = setTimeout(
    () => setToast((t) => t ? { ...t, leaving: true } : null), 4500
  );
  removeTimer.current = setTimeout(() => setToast(null), 5000);
}, []);

```

The two-timer pattern adds a `toast-leave` CSS class after 4.5 seconds to trigger a fade-out animation, then removes the element from the DOM after 5 seconds. Any component calls `const { showToast } = useToast()` to use it.

Screenshots

Login Page

PartTrack

Robot station parts inventory

EMAIL

e.g. j.smith@company.com

PASSWORD

.....

Sign in

Demo accounts

admin@parttrack.dev / admin123

alpha@parttrack.dev / delegate123

beta@parttrack.dev / delegate123

Parts Page

admin@parttrack.devADMINSign out

Sync with the database

PartsStationsUsersAlerts

Parts

All stationsAdd Part

4 below threshold

STATION	PART	SKU	SUPPLIER	MIN	QTY	IN TRANSIT	ADJUST	
Alpha Station	Bearing 6202	BRG-202	FastBear Ltd	20	81	—	0	Log ShipmentHistory
Alpha Station	Cognex 3000	IS99900	Cognex Corporation	5	5	—	0	Log ShipmentHistory
Alpha Station	Servo Motor 12V	SRV-001	RoboSupply Co	5	12	—	0	Log ShipmentHistory
Alpha Station	Timing Belt GT2	BLT-GT2	DriveWorks	10	10	—	0	Log ShipmentHistory
Beta Station	Arduino Mega	ARD-MEG	MicroElec	3	7	—	0	Log ShipmentHistory
Beta Station	Limit Switch NO	LMT-N01	SwitchPro	15	5	—	0	Log ShipmentHistory
Beta Station	Stepper NEMA17	STP-N17	MotorHub	4	2	—	0	Log ShipmentHistory

Stations Page

admin@parttrack.devADMINSign out

Sync with the database

PartsStationsUsersAlerts

Stations

Add Station

NAME	LOCATION	CREATED	
Alpha Station	Germany, Hamburg, Allee 28	06/05/2026	View Stock →Delete
Beta Station	Netherlands, Rotterdam, Waalhaven 12	06/05/2026	View Stock →Delete
Gamma Station	Poland, Warsaw, Pruszkowska 17	06/05/2026	View Stock →Delete

Users Page



admin@parttrack.devADMINSign out

Sync with the database

PartsStationsUsersAlerts

Users

+ Add User

EMAIL	ROLE	STATION	CREATED	
admin@parttrack.dev	ADMIN	—	06/05/2026	Delete
alpha@parttrack.dev	DELEGATE	Alpha Station	06/05/2026	Delete
beta@parttrack.dev	DELEGATE	Beta Station	06/05/2026	Delete

Alerts Page



admin@parttrack.devADMINSign out

Sync with the database

PartsStationsUsersAlerts

Alert Recipients

These addresses receive low-stock email alerts.

ADD RECIPIENT

e.g. manager@company.com

+ Add

EMAIL ADDRESS	ADDED	
vova.kireev@pm.me	20/05/2026	EditRemove

Add a new part

Add Part

STATION

Alpha Station



PART NAME

e.g. Cognex In-Sight 7000

SKU

e.g. IS7000-01

SUPPLIER

e.g. Cognex Corporation

MIN THRESHOLD

—

5

+

TRACKING LINK (OPTIONAL — IF ALREADY ORDERED)

<https://track.dhl.com/...>

Cancel

Add Part

Part history

PartTrack

admin@parttrack.dev ADMIN Sign out

Sync with the database

Parts Stations Users Alerts

← Back to Parts

Bearing 6202

BRG-282 · Alpha Station

All events

TIME	EVENT	USER	DETAILS
20/05/2026, 23:11:00	DELIVERED	admin@parttrack.dev	10 units Track → 20/05/2026
20/05/2026, 23:10:10	DELIVERED	admin@parttrack.dev	6 units Track → 20/05/2026
20/05/2026, 23:03:22	DELIVERED	admin@parttrack.dev	10 units Track → 20/05/2026
20/05/2026, 22:59:07	DELIVERED	admin@parttrack.dev	10 units Track → 20/05/2026
20/05/2026, 22:57:05	DELIVERED	admin@parttrack.dev	5 units Track → 20/05/2026
20/05/2026, 20:48:54	QTY -18	admin@parttrack.dev	58 → 40
20/05/2026, 20:46:32	QTY +26	alpha@parttrack.dev	32 → 58
20/05/2026, 20:46:23	QTY +2	alpha@parttrack.dev	30 → 32

Conclusions and further development

PartTrack was built to replace manual stock tracking in small manufacturing environments with an automated system that notifies the right people before a shortage causes a production stoppage. That goal was achieved. The application covers the full cycle: stock adjustments, shipment logging, delivery confirmation, and automated email alerts with progressive cooldowns. All of this runs from a single docker compose up command with no additional infrastructure.

The in-transit suppression logic turned out to be one of the more valuable design decisions. In early testing without it, alerts were firing for parts that already had a reorder in flight, which would have made the notification system annoying to use in practice. Adding the combined quantity + in_transit check resolved this completely and is also the most thoroughly tested part of the codebase.

The system is functional as it stands, but several areas could be improved in further development:

HTTPS support. The current setup serves traffic over plain HTTP, which is acceptable on a local network but would need TLS termination via a reverse proxy such as Caddy or nginx with Let's Encrypt before any internet-facing deployment.

Rate limiting. The login endpoint currently has no protection against brute force attempts. Adding express-rate-limit would address this with minimal effort.

Broader test coverage. The existing 22 tests cover authentication, input validation, and the alert logic. The delivery endpoint, station management, and admin user routes are not yet tested. Expanding the suite would increase confidence when making changes.

Mobile interface. The current UI works on narrow screens but was designed primarily for desktop use. A dedicated mobile layout would benefit delegates who check stock levels on the factory floor with a phone.

Refresh tokens. The current 8-hour JWT expiry is a reasonable compromise, but a proper refresh token mechanism would allow shorter-lived access tokens without forcing users to log in again mid-shift.

Multi-language support. Manufacturing facilities often employ staff across multiple languages. Internationalisation was not in scope for this project but would be a natural next step for a real deployment.

Overall, the project demonstrates that a meaningful gap exists between spreadsheet-based tracking and full ERP systems, and that it can be filled with a relatively compact, self-contained application. The core objective was met and the codebase is structured in a way that makes future extension straightforward.

References

1. **Node.js 20 Documentation** — <https://nodejs.org/docs/latest-v20.x/api/>
2. **Express 4 Documentation** — <https://expressjs.com/en/4x/api.html>
3. **PostgreSQL 16 Documentation** — <https://www.postgresql.org/docs/16/>
4. **React 18 Documentation** — <https://react.dev/>
5. **Docker Documentation** — <https://docs.docker.com/>